Haskell: The Purely Functional Programming Language

Haskell, an open source programming language, is the outcome of 20 years of research. Named after the logician, Haskell Curry, it has all the advantages of functional programming and an intuitive syntax based on mathematical notation. This second article in the series on Haskell explores a few functions.

Consider the function *sumInt* to compute the sum of two integers. It is defined as:

```
sumInt :: Int -> Int -> Int
sumInt x y = x + y
```

The first line is the type signature in which the function name, arguments and return types are separated using a double colon (::). The arguments and the return types are separated by the symbol (->). Thus, the above type signature tells us that the sum function takes two arguments of type *Int* and returns an *Int*. Note that the function names must always begin with the letters of the alphabet in lower case. The names are usually written in CamelCase style.

You can create a *Sum.hs* Haskell source file using your favourite text editor, and load the file on to the Glasgow Haskell Compiler interpreter (GHCi) using the following code:

```
$ ghci
GHCi, version 7.6.3: http://www.haskell.org/ghc/  :? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.

Prelude> :l Sum.hs
[1 of 1] Compiling Main             ( Sum.hs, interpreted )
Ok, modules loaded: Main.

*Main> :t sumInt
sumInt :: Int -> Int -> Int

*Main> sumInt 2 3
5
```

If we check the type of *sumInt* with arguments, we get the following output:

```
*Main> :t sumInt 2 3
sumInt 2 3 :: Int
```